

Głębokie badanie: Echo, Mobile, synchronizacja i plan dalszego rozwoju Ogarniacza

Zakres i wiarygodność analizy

Najważniejsze zastrzeżenie: **nie będę udawał, że wykonałem pełny audyt kodu repozytorium GitHub**. Próbowałem przejść przez podłączony GitHub i wyszukać repozytorium po nazwie projektu oraz charakterystycznych plikach (`KontekstRozmowyEcho.ts`, `NarzedziaEcho.ts`, `SilnikPrzypomnien.tsx`), ale dostęp do repozytoriów użytkownika wymagał w tej sesji interaktywnego uwierzytelnienia. Publiczne wyszukiwanie również nie znalazło indeksowanego repozytorium zawierającego te pliki. W efekcie mogłem dokładnie przeanalizować **logi Codexa, zakres zmian, nazwy plików, kolejne specyfikacje Etapów 6-9 i Mobile 3-4**, ale nie mogę niezależnie potwierdzić implementacji linia po linii.

Rekomendacje architektoniczne skonfrontowałem natomiast z aktualną dokumentacją Capacitor, Android, React oraz specyfikacją IndexedDB. Capacitor obecnie udostępnia osobne zdarzenia `resume`, `pause`, `appStateChange`, `appUrlOpen`, `getLaunchUrl()` oraz mechanizm przywracania wyników po ubiciu procesu Androida; oznacza to, że Mobile 4 można zrealizować bez tworzenia własnego systemu lifecycle. ¹ Capacitor ma również oficjalny `Network` plugin z `getStatus()` i `networkStatusChange`, więc do wykrywania offline/reconnect również nie jest potrzebna nowa abstrakcja platformowa, o ile aplikacja już używa Capacitor. ²

Na podstawie materiału, który podałeś, architektura wygląda dziś mniej więcej tak:

React/Vite UI → warstwa domenowa/repozytoria → IndexedDB → mechanizm synchronizacji → Android/Capacitor, z Echo jako klientem tej samej domeny, a nie osobnym systemem danych.

To jest dobry kierunek. Najważniejsza decyzja na teraz to **nie rozbudowywać dalej Echo funkcjonalnie, dopóki Mobile 4 nie ustabilizuje lokalnych danych, kolejki synchronizacji i lifecycle**.

Ocena dotychczasowych odpowiedzi Codexa

Echo Context

Odpowiedź z Etapu 8 jest architektonicznie sensowna. Codex nie stworzył osobnego silnika konwersacji, tylko rozszerzył istniejący `KontekstRozmowyEcho` o intencję, akcję oczekującą, etap doprecyzowania oraz wartości domyślne. To dokładnie właściwy poziom ingerencji dla tego etapu.

Szczególnie dobre są cztery decyzje widoczne w raporcie:

Obszar	Ocena
rolling context + structured state	bardzo dobry kierunek
jedna intencja przez kilka tur	konieczne

Obszar	Ocena
korekty typu „nie, czekaj...”	właściwa semantyka
jawne wartości przyjęte automatycznie	bardzo dobre UX
referencje „to / poprzedni”	potrzebne
osobne handlery dla każdej frazy	według raportu uniknięte

17/17 testów pokazuje przynajmniej, że scenariusze konwersacyjne zostały potraktowane poważniej niż zwykle testowanie pojedynczych funkcji. Nie jest to jednak niezależny dowód poprawności całej implementacji — jest to **deklaracja Codexa**, której bez kodu i wyników test runnera nie da się zweryfikować.

Mam jedną uwagę techniczną do zdania:

„upraszczam jeszcze rozpoznawanie referencji do tokenów i kategorii znaczeniowych”

To może być albo bardzo dobra refaktoryzacja, albo początek własnego mini-NLU opartego na heurystykach. Granica jest ważna. Echo powinno mieć niewielką warstwę normalizacji semantycznej, ale nie powinno zamieniać się w rosnący zbiór tokenów typu `poprzedni`, `wcześniejszy`, `ten`, `tamten`, `później`, `przesuń`. W Etapie 9 poprawnie podtrzymano zasadę, że różne sformułowania mają kończyć się na wspólnej operacji domenowej.

Echo Domain Tools

Etap 9 również idzie we właściwą stronę. Najistotniejsza część raportu brzmi:

„Dodano wyszukiwanie, odczyt, edycję i miękkie usuwanie przez istniejące repozytoria.”

To jest ważniejsze niż sama liczba obsługiwanych komend. Echo **nie może stać się drugim systemem CRUD**. Jeżeli `NarzedziaEcho.ts` jest cienką warstwą nad repozytoriami/domeną, architektura jest dobra.

Największe zastrzeżenie dotyczy walidacji. Po Etapie 8 raport mówił o **17 testach**, po znacznie większym Etapie 9 o **19 testach**. Czyli licznik wzrósł tylko o dwa, podczas gdy zakres wzrósł o read, search, edit, reschedule, delete, confirmation i context follow-up. Nie oznacza to automatycznie braku pokrycia — jeden test może zawierać kilka kroków — ale jest to sygnał, że przy najbliższej pracy nad Echo warto sprawdzić **testy kontraktowe operacji**, a nie dodawać dziesiątki kolejnych testów językowych.

Nie testowałbym 40 wariantów:

„przenieś”, „przesuń”, „daj później”, „zmień termin”...

Lepiej sprawdzić kilka reprezentatywnych wariantów i przede wszystkim to, że wszystkie kończą się na **tej samej komendzie domenowej**.

Powiadomienia Mobile

Mobile 3 wygląda na najlepszy jakościowo z pokazanych etapów, ponieważ Codex znalazł trzy konkretne luki, poprawił je i uruchomił szerszą walidację:

- akcje Androida,
- aktualizację rekordu po akcji,
- propagację zmienionego terminu do powiadomienia.

Kierunek dotyczący exact alarmów jest zgodny z aktualnymi zaleceniami Androida. Android wprost zaleca inexact alarms dla większości przypadków i exact alarms tylko wtedy, gdy precyzyjna chwila jest centralna dla funkcji aplikacji, np. alarmu lub kalendarza. Exact alarms kosztują więcej energii i na Androidzie 12+ wymagają dodatkowego dostępu. ³

To sprawia, że decyzja Codexa:

„sama eskalacja nie będzie już wystarczać — dokładność pozostanie tylko dla krytycznego, bezpośredniego terminu absolutnego”

jest **merytorycznie bardzo dobra**.

Jest tu również istotny szczegół przyszłościowy: aktualna dokumentacja Capacitor Local Notifications v8 pozwala jawnie wskazać `isExactNotification`; zwykle przypomnienia mogą być planowane jako nieprecyzyjne, a plugin obsługuje również `update()`, `cancel()`, akcje oraz listener `localNotificationActionPerformed`. ⁴ Nie należy jednak teraz aktualizować Capacitor tylko po to, żeby skorzystać z tych API — najpierw trzeba sprawdzić wersję faktycznie używaną przez repo.

Permission flow z Twojej specyfikacji również jest poprawny. Od Androida 13 `POST_NOTIFICATIONS` jest runtime permission, a przy targetowaniu Androida 13+ aplikacja sama kontroluje moment wyświetlenia prośby. Dokumentacja Androida zaleca wykorzystanie tego do wyjaśnienia użytkownikowi, dlaczego funkcja potrzebuje powiadomień, zamiast bezkontekstowego promptu przy pierwszym uruchomieniu. ⁵

Natomiast w przedstawionym logu jest jedna **realna luka dowodowa**. Codex napisał:

„Teraz sprawdzam produkcyjny build i synchronizację projektu Android.”

Po czym wklejony materiał kończy się informacją o sześciu zmienionych plikach. Nie ma końcowego raportu mówiącego jednoznacznie:

```
Build: PASS
cap sync: PASS
Android build: PASS
```

Dlatego Mobile 3 uznałbym za:

logika/testy/typecheck — potwierdzone przez raport Codexa; końcowy build Android — do domknięcia.

Nie ma sensu robić ponownego audytu Mobile 3. Wystarczy przy początku następnego etapu wykonać jeden krótki gate buildowy.

Spójność historii zmian

Jest jeszcze drobna rozbieżność w historii.

Po Etapie 8 Codex napisał:

„Nie wykonano commita ani push.”

Po Etapie 9 również nie ma informacji o commitowaniu.

Przed Mobile 3 mówi natomiast:

„Preflight jest czysty (`main` ma tylko jeden lokalny commit względem `origin/main`, brak zmian roboczych).”

Najbardziej prawdopodobne jest więc, że zmiany Echo zostały między tymi etapami lokalnie zacommitowane. Nie jest to problem techniczny, ale pokazuje, dlaczego w kolejnych promptach warto wymagać wyłącznie krótkiego:

```
git status --short  
git log -1 --oneline
```

zamiast ponownych analiz historii repo.

Mobile Offline i niezawodność danych

Mobile 4 powinien teraz dostać absolutny priorytet.

Nie dlatego, że Echo wymaga przebudowy. Wręcz przeciwnie: Echo jest już wystarczająco funkcjonalne, żeby ujawnić problem, gdy pod spodem dane lub sync są niestabilne.

Po Etapie 9 użytkownik może już przez Echo:

```
create → read → search → reschedule → delete
```

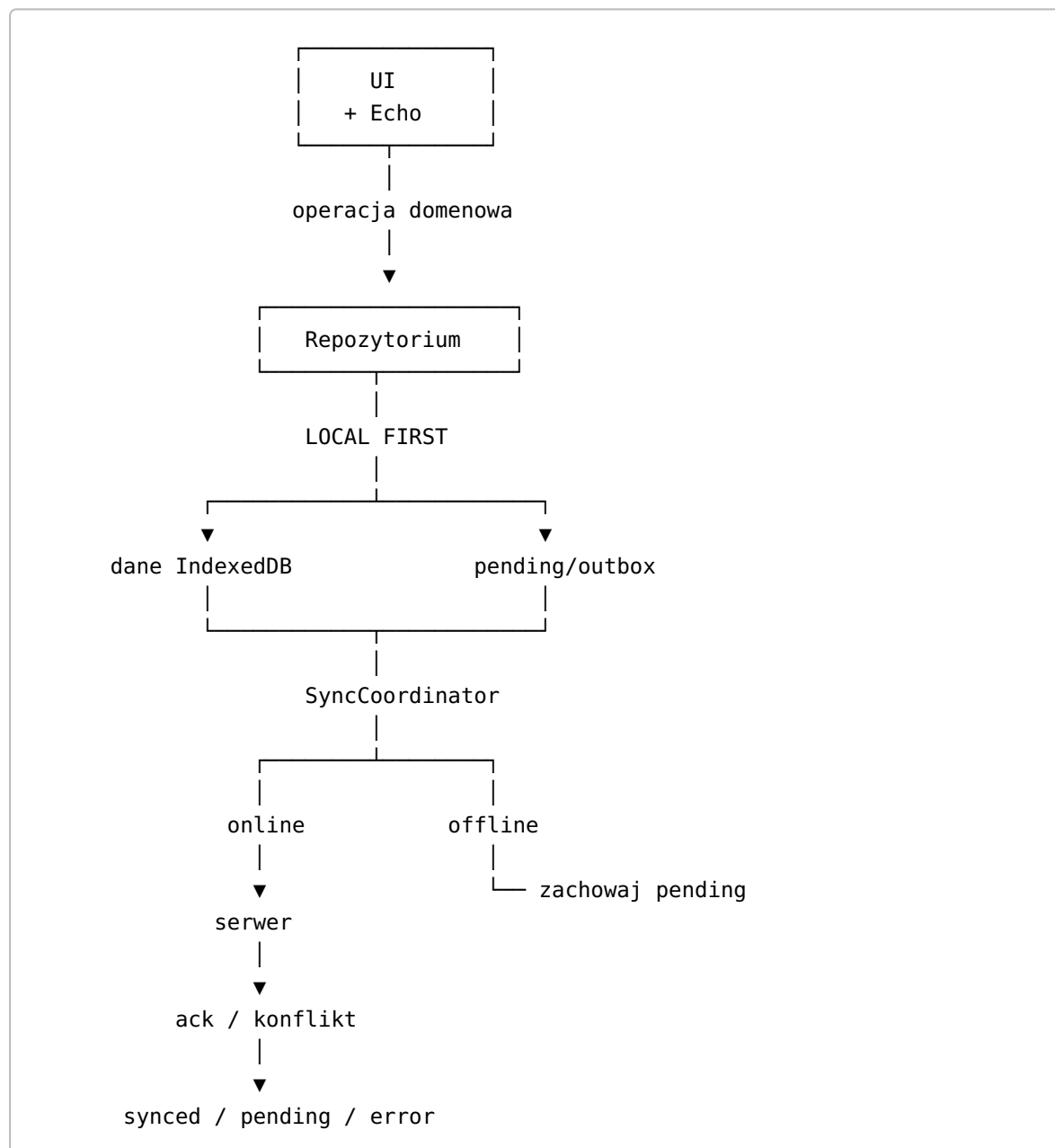
a po Mobile 3 zmiany terminów wpływają również na powiadomienia. To oznacza, że przy utracie sieci jedna operacja użytkownika może dotyczyć:

lokalnego rekordu → kolejki synchronizacji → powiadomienia Android → statusu synchronizacji → kontekstu Echo.

Jeżeli teraz dodasz kolejne funkcje, zanim ustabilizujesz ten łańcuch, liczba możliwych stanów błędnych szybko wzrośnie.

Model, do którego powinien dążyć Mobile 4

W praktyce właściwy model wygląda tak:



Najważniejszą zasadą powinno być:

synchronizacja jest skutkiem zapisu lokalnego, a nie warunkiem zapisu lokalnego.

IndexedDB został zaprojektowany właśnie dla trwałych danych aplikacji działających także offline i zapewnia transakcyjne operacje na lokalnej bazie. Specyfikacja pokazuje, że operacje w transakcji są zatwierdzane razem, a abort wycofuje zmiany. ⁶

Jeżeli obecna warstwa IndexedDB pozwala objąć jedną transakcją rekord oraz istniejący wpis synchronizacji, najlepiej utrwalić:

```
zmiana encji
+
informacja "należy zsynchronizować"
```

atomowo. Nie powinno być sytuacji:

```
zapisano zadanie
→ proces Androida został ubity
→ wpis do kolejki jeszcze nie powstał
→ zadanie istnieje lokalnie, ale nigdy nie dotrze na serwer
```

Nie oznacza to budowy event sourcingu. Wystarczy istniejący mechanizm pending/outbox rozszerzony tak, aby zapis był trwały.

IndexedDB i cold start

Nie należy zakładać, że poprzednia instancja JavaScript jeszcze istnieje. Capacitor na Androidzie mapuje `resume` na lifecycle natywnej Activity, a dokumentacja dodatkowo ostrzega, że system może zabić proces aplikacji z powodów pamięciowych. ¹

Cold start powinien więc wyglądać następująco:

```
start procesu
↓
otwórz IndexedDB / migracje
↓
odtwórz lokalny model
↓
ustal początkowy route / sourceRef / launch URL
↓
pokaż lokalne dane
↓
sprawdź network
↓
uruchom synchronizację asynchronicznie
```

Nie:

```
start
↓
czekaj na Internet
↓
synchronizuj
↓
dopiero pokaż aplikację
```

Capacitor udostępnia `getLaunchUrl()` właśnie dla cold-start deep links oraz `appUrlOpen` dla kolejnych otwarć linków w żyjącej aplikacji. ¹

To jest też dobry moment, aby sprawdzić scenariusz:

aplikacja ubita → użytkownik klika powiadomienie → aplikacja startuje → otwiera właściwy rekord.

To jest bardziej wartościowy test niż kolejne testowanie samego parsera Echo.

Resume bez lawiny synchronizacji

Jedna z najważniejszych rzeczy w Mobile 4 nie jest wprost wypisana w pierwotnym promptcie:

musi istnieć co najwyżej jeden aktywny sync danego zakresu.

Źródła wywołania będą co najmniej trzy:

```
cold start
resume
networkStatusChange: offline → online
```

Do tego może dojść ręczne odświeżenie.

Nie mogą one powodować:

```
resume → sync A
network reconnect → sync B
React effect remount → sync C
```

React w trybie development celowo uruchamia dodatkową sekwencję setup/cleanup dla efektów, aby wykrywać błędy subskrypcji. Listener bez poprawnego cleanup może przez to ujawnić podwójne rejestracje, a analogiczne problemy mogą wystąpić w produkcji wskutek remountów. ⁷

Dlatego `Network.addListener(...)`, `App.addListener(...)` i podobne subskrypcje powinny mieć jednoznaczny lifecycle i cleanup. Capacitor oficjalnie wspiera `networkStatusChange`, więc nie ma potrzeby pollingować stanu połączenia. ²

Praktycznie wystarczy niewielki mechanizm:

```
if (syncInFlight) {
  return syncInFlight;
}

syncInFlight = wykonajSync().finally(() => {
  syncInFlight = null;
});
```

```
return syncInFlight;
```

albo jego istniejący odpowiednik w projekcie.

Nie budowałbym teraz schedulera, WorkManagera ani Background Runnera, jeżeli repo nie używa ich już do synchronizacji.

Status synchronizacji

Cztery statusy z Twojej specyfikacji są wystarczające:

Stan	Znaczenie
synced	brak oczekujących lokalnych zmian
pending	lokalne zmiany czekają na synchronizację
offline	urządzenie jest offline; lokalna praca nadal możliwa
error	sync został wykonany, ale zakończył się błędem wymagającym retry/interwencji

Stan `offline` należy ustalać przy pomocy rzeczywistego stanu sieci, np. Capacitor Network, nie na podstawie błędu pojedynczego requestu. ²

Nie wprowadzałbym użytkownikowi:

```
retry #4,  
queue=7,  
revision mismatch,  
HTTP 409,  
lastCursor=...
```

To są dane diagnostyczne, nie produktowe.

Trwałość IndexedDB

IndexedDB jest właściwym mechanizmem dla tej architektury; specyfikacja definiuje go jako lokalną bazę wykorzystującą trwałe struktury i transakcje. ⁸ Specyfikacja wymaga również, aby implementacje radziły sobie ze starszymi formatami serializacji przy zmianie implementacji przeglądarki, co jest ważne w kontekście aktualizacji silnika WebView. ⁹

Jednocześnie webowe storage nie powinno być traktowane jako jedyna kopia zapasowa danych. Dane przeglądarkowe są domyślnie klasyfikowane jako best-effort, a mechanizmy i kryteria usuwania zależą od silnika. ¹⁰ Dla Ogarniacza oznacza to praktycznie:

IndexedDB = źródło prawdy podczas lokalnej pracy i offline.

Synchronizacja = trwała druga kopia i mechanizm wielourzędzeniowy.

Nie przenosiłbym aplikacji teraz do SQLite tylko z powodu Mobile 4.

Priorytetowy plan kolejnych wdrożeń

Moja kolejność jest wyraźnie inna niż „dodawajmy następne możliwości Echo”. Funkcjonalnie Ogarniacz jest już wystarczająco szeroki. Najwięcej wartości daje teraz **zamykanie pętli niezawodności**.

Priorytet	Etap	Co dostarcza	Dlaczego teraz
P0	domknięcie Mobile 3	jeden build + Android sync/ smoke	brak końcowego wyniku w pokazanym logu
P0	Mobile 4 Lifecycle/ Offline	cold start, resume, durable pending queue, reconnect	fundament bezpieczeństwa danych
P0	integracja Domain → Sync → Notifications	gwarancje po create/edit/ delete	zapobiega sierotom i niespójności
P1	Mobile Reliability	process death, notification cold start, migrations	prawdziwe użycie Androida
P1	Sync UX	4 statusy, retry, bez technicznego logu	użytkownik wie, czy dane są bezpieczne
P1	Echo 2.1 hardening	niejednoznaczność i kontrakty domain tools	poprawa jakości bez nowych intentów
P2	Release hardening	permission denial, upgrade, storage errors	przygotowanie do codziennego użycia
P3	kolejne feature'y	dopiero później nowe możliwości	nie zwiększać powierzchni błędów

Domknięcie Mobile 3

Nie rób ponownego audytu.

Wystarczy:

```
git status --short
npm run typecheck
npm run build
npx cap sync android
```

oraz jeden krótki smoke scenariusz na emulatorze/telefonie:

```
utwórz przypomnienie
→ zmień termin
→ sprawdź, że stary alarm nie istnieje
→ sprawdź nowy
→ usuń rekord
→ sprawdź brak pending notification
```

Jeżeli projekt ma już automatyczny Android build w npm/Gradle, można zamiast dodatkowego ręcznego testu użyć istniejącej komendy.

Mobile 4 jako najbliższe właściwe wdrożenie

Zakres powinien obejmować wyłącznie:

```
cold start
resume
offline write
durable pending queue
reconnect flush
single-flight sync
existing conflict policy
sync status
```

Nie dorzucać przy okazji:

- background sync po wielu godzinach,
- service workera,
- WorkManagera,
- nowego storage,
- historii synchronizacji,
- analytics,
- rozbudowanego retry UI.

Integracja po Mobile 4

To powinien być bardzo mały etap, ale o dużej wartości.

Sprawdzić następujący łańcuch:

```
Echo: "przypomnij jutro"
  ↓
Repozytorium
  ↓
IndexedDB
  ↓
pending sync
  ↓
NotificationService
```

Potem:

```
Echo: "jednak godzinę później"
  ↓
ten sam rekord
  ↓
```

```
nowa data
  ↓
stare notification anulowane / zaktualizowane
  ↓
jedna nowa synchronizacja
```

I:

```
Echo: "usuń to"
  ↓
soft delete / właściwa operacja domenowa
  ↓
powiadomienie anulowane
  ↓
tombstone/pending sync zachowany aż do ACK
```

To ostatnie jest szczególnie ważne: **lokalnie usuniętego rekordu nie wolno usuwać również z kolejki synchronizacyjnej, zanim serwer nie dowie się o usunięciu**. W przeciwnym razie po kolejnej synchronizacji rekord może wrócić.

Mobile Reliability

Dopiero potem warto zrobić etap obejmujący nietypowe zachowania Androida:

```
process killed
notification opens cold app
permission denied
exact alarm denied
WebView/app upgrade
IndexedDB schema migration
sync interrupted halfway
```

Capacitor sam zwraca uwagę, że Android może ubić proces podczas używania zewnętrznych Activity i udostępnia `appRestoredResult` do obsługi wyników po ponownym uruchomieniu. ¹ Nie znaczy to, że trzeba teraz implementować pełne „session restoration”; należy po prostu przestać zakładać, że pamięć JS jest trwała.

Echo później

Echo 2.0 ma już właściwy podstawowy zestaw:

```
Create
Read
Search
Edit
Reschedule
Delete
```

Context follow-up
Confirmation

Nie widzę dziś uzasadnienia dla Etapu 10 typu:

„dodaj następne 15 intentów”.

Bardziej wartościowe będą później trzy ulepszenia:

deterministyczne rozstrzygnięcie niejednoznaczności, np. dwa zadania „mechanik”;

kontrakty domain tools, aby parser języka i wykonanie operacji pozostawały rozdzielone;

spójność z offline, aby lokalne odczyty/search/edit działały bez sieci, jeżeli wszystkie wymagane dane istnieją lokalnie.

Prompt dla Codexa do Mobile 4 bez przepalania kredytów

Poniższa wersja jest w mojej ocenie lepsza od obecnej specyfikacji, bo ogranicza eksplorację repo i dokładnie wskazuje, gdzie nie wolno budować nowych systemów.

ETAP MOBILE 4 – LIFECYCLE, OFFLINE I NIEZAWODNOŚĆ DANYCH

Nie wykonuj pełnego audytu repo.

Nie wracaj do Echo.

Nie przebudowuj storage ani synchronizacji od zera.

CEL

Aplikacja ma zachować lokalną pracę i dane przy:

- cold start,
- resume,
- braku internetu,
- odzyskaniu internetu,
- normalnym ubiciu procesu Android/WebView.

Najpierw sprawdź wyłącznie istniejące punkty wejścia dotyczące:

- IndexedDB / repozytoriów,
- obecnego mechanizmu synchronizacji i conflict resolution,
- inicjalizacji aplikacji,
- Capacitor App/Network,
- statusu synchronizacji.

Nie czytaj całego repo, jeżeli zależności tych modułów tego nie wymagają.

IMPLEMENTACJA

1. LOCAL FIRST

Operacje domenowe zapisują się lokalnie niezależnie od dostępności sieci.

Brak internetu nie może powodować odrzucenia poprawnego lokalnego:
create / edit / delete.

Nie duplikuj persistence.

2. COLD START

Po uruchomieniu nowej sesji JS:

- otwórz istniejące local storage/IndexedDB,
- odtwórz wymagany stan,
- zainicjalizuj routing/deep link/sourceRef,
- pokaż dane lokalne,
- dopiero potem rozpocznij sync w tle.

Nie uzależniaj startu UI od dostępności backendu.

Nie zakładaj istnienia stanu poprzedniej sesji JS.

3. RESUME

Użyj istniejącego mechanizmu lifecycle.

Resume:

- nie resetuje bieżącej trasy,
- nie przeładowuje całej aplikacji,
- uruchamia tylko potrzebne odświeżenie/sync.

Nie rejestruj wielokrotnie listenerów.

4. SINGLE-FLIGHT SYNC

Cold start, resume i reconnect mogą żądać synchronizacji,
ale nie mogą uruchomić kilku równoległych sync tego samego zakresu.

Wykorzystaj istniejący coordinator/lock, jeśli istnieje.
Jeżeli go nie ma, dodaj minimalny single-flight guard.

5. OFFLINE QUEUE

Rozbuduj istniejący mechanizm pending/queue.

Niezsynchronizowana operacja musi przetrwać restart aplikacji.

Nie twórz drugiej kolejki.

Nie usuwaj operacji pending przed potwierdzonym powodzeniem synchronizacji.

Jeżeli obecna architektura pozwala, zapis rekordu i oznaczenie go jako

pending
powinny nastąpić w tej samej lokalnej transakcji.

6. RECONNECT

Po przejściu offline -> online:

- wykonaj pending sync,
- bez duplikowania operacji,
- bez wielokrotnego wykonywania tego samego delete/create/update.

Wykorzystaj istniejące ID/revision/idempotency mechanizmy.

Nie projektuj nowego event sourcingu.

7. KONFLIKTY

Zachowaj istniejącą politykę conflict resolution.

Nie twórz CRDT.

Modyfikuj mechanizm konfliktów tylko wtedy, gdy konkretny test lifecycle ujawni błąd.

8. STATUS

UI ma rozróżniać maksymalnie:

synced
pending
offline
error

Nie pokazuj użytkownikowi technicznych logów ani numerów retry.

9. INDEXEDDB

Nie zmieniaj technologii storage.

Sprawdź tylko, czy dane i pending sync wymagane lokalnie nie są trzymane wyłącznie w React state / pamięci JS.

10. NIE DODAWAJ

- nowego schedulera,
- WorkManagera, jeżeli obecna architektura go nie potrzebuje,
- Background Runnera,
- Service Workera tylko dla tego etapu,
- nowego backend API,
- nowego conflict model,
- historii synchronizacji,
- zmian w Echo,

- dużej refaktoryzacji repozytoriów.

TESTY

Podczas implementacji uruchamiaj tylko testy zmienianych modułów.

Dodaj minimalne scenariusze potwierdzające:

- A. zapis lokalny offline + pending
- B. pending przetrwa ponowne otwarcie storage
- C. reconnect wysyła pending
- D. resume + reconnect nie uruchamiają dwóch równoległych sync
- E. istniejąca polityka konfliktów nadal działa
- F. status przechodzi poprawnie:
synced / pending / offline / error

Nie twórz dziesiątek wariantów testów.

Po zakończeniu:

- uruchom targeted tests,
- pełny test suite dokładnie raz, ponieważ zmieniany jest centralny data/sync layer,
- npm run typecheck,
- npm run build.

Jeżeli projekt posiada istniejącą szybką komendę Android sync/build, uruchom ją raz na końcu.

Nie ponawiaj buildów po każdej małej zmianie.

RAPORT

ETAP MOBILE 4 – LIFECYCLE / OFFLINE

Cold start: PASS / FAIL
Resume: PASS / FAIL
Offline local write: PASS / FAIL
Persistent pending queue: PASS / FAIL
Reconnect: PASS / FAIL
No duplicate sync: PASS / FAIL
Conflict policy: PASS / FAIL
Sync status: PASS / FAIL

Targeted tests: PASS / FAIL – X tests
Full tests: PASS / FAIL – X tests
Typecheck: PASS / FAIL
Build: PASS / FAIL
Android sync/build: PASS / FAIL / NOT APPLICABLE

Zmodyfikowane pliki:

- ...

Znane ograniczenia:
maksymalnie 3 krótkie punkty.

Nie wykonuj commita ani push.
STOP.

Ten prompt daje Codexowi dużo mniej przestrzeni na kilkuminutowe „badanie wszystkiego”, a jednocześnie nie narzuca konkretnej implementacji, zanim agent zobaczy istniejący system.

Jak prowadzić kolejne etapy taniej i bez „AI-driven overengineering”

Największe ryzyko dalszej pracy nie polega obecnie na zbyt małej liczbie funkcji. Jest nim **powtarzanie szerokiego audytu repo przed każdym małym etapem**.

Schemat powinien zostać zmieniony z:

```
zbadaj repo
→ zbadaj architekturę
→ przejrzyj historię
→ znajdź wszystkie potencjalne problemy
→ implementuj
→ dużo testuj
→ pełny test suite
→ lint
→ build
→ android sync
```

na:

```
git status
↓
otwórz znane moduły wejściowe
↓
podążaj tylko za wymaganymi zależnościami
↓
implementacja
↓
targeted tests
↓
jedna walidacja końcowa
```

Jeden audit na kamień milowy, nie na prompt

Etapy 8 i 9 dotyczą tego samego pionowego slice Echo. Nie ma sensu ponownie audytować Echo przy każdej kolejnej operacji.

Podobnie Mobile 3 i Mobile 4 korzystają z tego samego fundamentu:

```
Repozytorium  
NotificationService  
platform  
sync  
IndexedDB
```

Po Mobile 4 warto zrobić **jeden** szerszy milestone review obejmujący integrację mobilną. Potem znowu kilka etapów bez pełnego audytu.

Testować inwarianty, nie wszystkie zdania użytkownika

Dla Echo warto utrzymać kilka naprawdę ważnych inwariantów:

```
język użytkownika  
↓  
semantic decision  
↓  
domain operation  
↓  
ten sam rekord
```

oraz:

```
brak pewności  
→ brak mutacji  
→ jedno pytanie
```

i:

```
destructive + ambiguous/mass  
→ confirmation
```

To da większą wartość niż 100 testów synonimów.

Warstwa danych zasługuje na więcej testów niż UI

W Mobile 4 nie przesadzałbym z testami komponentów React.

Najcenniejsze testy dotyczą:

```
write offline  
queue durability  
retry
```

```
deduplication
conflict
delete/tombstone
notification scheduling consistency
```

Ponieważ to od nich zależy utrata danych.

Listener lifecycle traktować jako kontrakt

Każda subskrypcja:

```
App.addListener(...)
Network.addListener(...)
LocalNotifications.addListener(...)
```

powinna mieć jasno określone miejsce rejestracji i cleanup. React zaleca, aby cleanup dokładnie odwracał setup efektu; Strict Mode celowo wykonuje dodatkowy cykl, żeby ujawnić niepoprawne subskrypcje. ¹¹

To jest bardzo tani sposób zapobiegania późniejszemu „duchom”: podwójnym sync, podwójnym deep linkom czy podwójnym akcjom powiadomień.

Nie przenosić danych do innej technologii

Mobile 4 nie jest argumentem za migracją do SQLite. IndexedDB wspiera transakcyjne lokalne dane i jest przeznaczony m.in. dla zaawansowanych aplikacji offline. ⁸

Zmiana storage miałaby teraz bardzo wysoki koszt:

```
migracja
adaptery
testy
konflikty
backup
wersjonowanie
Android/iOS/web divergence
```

bez rozwiązania głównego problemu, którym jest najprawdopodobniej lifecycle koordynatora synchronizacji.

Nie dodawać background sync przed rozwiązaniem foreground sync

Najpierw aplikacja powinna być niezawodna w prostym modelu:

```
offline
→ lokalna praca
→ aplikacja pozostaje/zostaje uruchomiona
```

```
→ reconnect/resume
→ flush
```

Dopiero jeżeli badania użytkowe pokażą, że konieczna jest synchronizacja przy długo zamkniętej aplikacji, można rozważać natywny background execution.

Exact alarms utrzymać jako wyjątek

Obecna decyzja z Mobile 3 jest zgodna z Androidem: zwykłe przypomnienie powinno być inexact, a alarm dokładny tylko tam, gdzie precyzyjna godzina ma rzeczywiste znaczenie dla użytkownika. ³

Przy ewentualnym przejściu na aktualną wersję Capacitor Local Notifications trzeba uważać, ponieważ obecna dokumentacja opisuje `isExactNotification` jako domyślnie `true`; dla zwykłych przypomnień należałoby więc jawnie wybrać tryb nieprecyzyjny. ⁴

Werdykt

Na podstawie dostępnych zmian **nie cofałbym obecnej architektury i nie rozpoczynałbym kolejnego dużego audytu.**

Etapy 8 i 9 wyglądają jak sensowny rozwój Echo:

```
Etap 6   vertical slice
      ↓
Etap 8   conversation context
      ↓
Etap 9   domain operations
```

Mobile 3 właściwie przesunął odpowiedzialność powiadomień bliżej warstwy danych i rozwiązał szczególnie ważny problem aktualizacji/usuwania alarmów. Kierunek exact-alarm policy jest zgodny z Androidem. ¹²

Największą wartość produktową daje teraz Mobile 4.

Docelowa kolejność prac powinna być:

```
domknąć build Mobile 3
      ↓
MOBILE 4
local-first + lifecycle + durable queue + reconnect
      ↓
integracja
Echo ↔ Repo ↔ Sync ↔ Notifications
      ↓
process-death / upgrade / permission reliability
      ↓
release hardening
```

↓
dopiero potem dalsze rozszerzanie Echo

Najważniejsza zasada architektoniczna dla kolejnych etapów brzmi:

UI, Echo, powiadomienia i synchronizacja nie powinny mieć własnych sposobów zmieniania danych. Wszystkie powinny przechodzić przez tę samą logikę domenową/repozytoryjną.

A najważniejsza zasada dla Codexa:

Nie pytaj go za każdym razem „co jest nie tak z całym repo?”. Daj mu jeden pionowy cel, znane punkty wejścia, kilka inwariantów oraz jeden końcowy gate.

W obecnym stanie projektu jest to prawdopodobnie szybsza droga do stabilnej aplikacji niż następne setki linii nowych funkcji.

1 App Capacitor Plugin API | Capacitor Documentation

https://capacitorjs.com/docs/apis/app?utm_source=chatgpt.com

2 Network Capacitor Plugin API

https://capacitorjs.com/docs/apis/network?utm_source=chatgpt.com

3 12 Schedule alarms | Background work

https://developer.android.com/develop/background-work/services/alarms?utm_source=chatgpt.com

4 Local Notifications Capacitor Plugin API

https://capacitorjs.com/docs/apis/local-notifications?utm_source=chatgpt.com

5 Notification runtime permission | Jetpack Compose

https://developer.android.com/develop/ui/compose/notifications/notification-permission?utm_source=chatgpt.com

6 8 9 Indexed Database API 3.0

<https://www.w3.org/TR/IndexedDB-3/>

7 11 useEffect

https://react.dev/reference/react/useEffect?utm_source=chatgpt.com

10 Storage quotas and eviction criteria - Web APIs | MDN

https://developer.mozilla.org/en-US/docs/Web/API/Storage_API/Storage_quotas_and_eviction_criteria